

Engine poboru danych dla rzetelnej analizy sytuacji w Iranie i postępów procesów kaskadowych

Streszczenie wykonawcze i potrzeby informacyjne

Aby zaprojektować **obronny metodologicznie i wdrażalny technicznie** engine poboru danych (OSINT + otwarte statystyki) dla Iranu oraz do zasilania modeli kaskad (SD / sieci), badacz musi doprecyzować kilka „twardych” informacji (to są kluczowe niewiadome wejściowe, które determinują architekturę, koszty i replikowalność):

1. **Zakres zmiennych modelu** do kalibracji (minimalny zestaw) oraz ich semantyka: czy dana zmienna to *stan pozytywny* (wyżej=lepiej), czy *stres* (wyżej=gorzej).
2. **Źródła pierwotne vs wtórne** dla każdej hipotezy (zasada 2 niezależnych proxy): które źródło jest „primary”, które służy walidacji/triangulacji.
3. **Wymagany horyzont i częstotliwość** (u Ciebie: tydzień jako krok SD) oraz polityka: *step-hold vs interpolacja* dla danych miesięcznych/rocznych (w Twoim standardzie: domyślnie step-hold).
4. **Dostęp i licencje**: które źródła wymagają kont/tokenów (np. ACLED, Cloudflare Radar, UN Comtrade v1, UNHCR API), jakie są ograniczenia rate-limit i reuse. ¹
5. **Polityka replikowalności** („reżim absolutny”): jak wersjonujesz kod, konfiguracje, snapshoty danych (raw/clean/transformed/normalized/final), oraz manifest uruchomień (run-manifest).
6. **Tryb użycia**: dashboard + analiza odporności i dynamiki kryzysu (bez taktyki/operacyjności). To determinuje granice etyczne i dual-use.

Co zrobiłem w tym researche (jawnie, zgodnie z Twoją dyspozycją)

Najpierw przeszukałem wskazane repozytoria GitHub (konektor GitHub), aby znaleźć istniejące wzorce: (a) kontrakty danych/artefaktów, (b) wzorce CI/uruchamialności, (c) istniejącą warstwę symulacji (Twoje MC/Morris/Sobol), (d) potencjalne załączki ingestion. Następnie wykonałem web research pod oficjalne API/metodologie źródeł danych (ACLED, World Bank, OPEC, OFAC, UNHCR, Cloudflare Radar, OONI, IODA/CAIDA, UN Comtrade). ²

Wynik przeszukania repozytoriów (najważniejsze pliki i wnioski)

Kluczowe znaleziska dla budowy engine’u poboru danych:

- **DonkeyJLove/SymulacjaKaskadySieciowej**: repo ma kompletną warstwę SD + MC + GSA (Morris/Sobol) oraz krok tygodniowy i out-of-the-box konfigurację. To jest naturalne miejsce „podpięcia” danych empirycznych. ³
- Konfiguracja runtime i walidacja typów/kluczy w `config.py` – warto skopiować idiom (default → YAML → CLI override → coercion → walidacja) do engine’u danych. ⁴
- `gsa_common.py` zawiera jawnie zdefiniowane rozkłady parametrów na hipersześcianie i transformacje kwantylowe (beta/truncnorm), co jest wzorcem „jawnych definicji” także dla danych (transform/normalization spec). ⁵

- **DonkeyJLove/sbom**: repo zawiera bardzo przydatne wzorce „reżimu absolutnego” dla danych jako zdarzeń:
- `AID_CONTRACT.md` – minimalny kontrakt tożsamości (AID) spinający byt, pomiar, korelację i artefakty; ten idiom można bezpośrednio „przenieść” na byt: `dataset/run` oraz `country/variable`.⁶
- `docs/04_DATA_MODEL.md` – normatywny model „koperty zdarzenia” (`@timestamp`, `event_type`, `aid`, `payload`) i idea: „indeksujemy to, co potrzebne do wnioskowania” oraz „delta ważniejsza niż stan”. To jest idealny szkielet dla logowania ingest/transform/normalize wraz z metadanymi.⁷
- `środowiska-testowe/docker-compose.lab.yml` – gotowy wzorzec reproducibile lab (Docker, profile, healthcheck), który możesz skopiować/uprościć do data-engine (np. DuckDB/MinIO/Elastic jako opcje).⁸
- **DonkeyJLove/glitchlab**: repo ma wzorzec *powtarzalnych artefaktów analitycznych* i deterministycznego zapisu wyników (np. `.glx/.../metrics.json`) oraz implementacje metryk grafowych (pagerank, betweenness approx, LPA) – przydatne, jeśli Twoje „kaskady” obejmują sieci/wykresy zależności i chcesz je automatycznie mierzyć z danych.⁹

Wniosek repo-scan: **nie ma gotowego, dedykowanego engine’u poboru danych dla Iranu**, ale masz bardzo silne komponenty-wzorce (kontrakt danych, lab/docker, config/CLI, artefakty analityczne). Engine należy dobudować jako *moduł* repozytorium SymulacjaKaskadySieciowej (albo jako podrepo), zgodnie z Twoim „STANDARD DANYCH”.

Założenia metodologiczne: reżim naukowy absolutny i Twój standard danych

W tym projekcie Twoim „prawem danych” jest dostarczony **STANDARD DANYCH** (pipeline i reguły normalizacji do 0–1, klasy A/B/C, reżimy braków/outlierów, jawne metadane, równoległe warstwy raw/clean/transformed/normalized/final). Traktuję go jako **normatywną specyfikację projektu** (czyli „tak ma być”), a engine ma go egzekwować automatycznie.

Żeby to spełnić w logice „reżimu naukowego absolutnego”, engine musi mieć nie negocjowalne własności:

Transparentność

Każdy dataset/seria ma: (a) spec źródła i wersję poboru, (b) jawne transformacje, (c) jawne normalizacje, (d) jawne flagi imputacji i jakości, (e) jawny kierunek semantyczny (*higher_is_worse* vs *higher_is_better*). (To dokładnie odpowiada Twojemu wymogowi: „nie trzymasz tylko znormalizowanej wersji” i „nie mieszasz konwencji bez pola semantic_direction”.)

Replikowalność

Replikowalność dotyczy dwóch poziomów: - *kod/środowisko*: pinned zależności + Docker + lockfile + wersja Pythona + identyfikator commita;

- *dane*: surowe snapshoty (raw bytes), checksums, request-params, timestamp poboru, oraz manifest uruchomień.

Wzorzec AID + koperty zdarzeń z repo `sbom` jest gotowym przenośnym idiomem na te potrzeby.⁶

⁷

Jawne definicje dostępu i ograniczeń źródeł

- ACLED wymaga konta i procesu uwierzytelniania (cookie/OAuth), ma limit odpowiedzi i zaleca paginację. ¹⁰
- World Bank Indicators API (v2) nie wymaga klucza i ma jasno opisaną strukturę wywołań. ¹¹
- OPEC MOMR i załączniki tabelaryczne są publikowane cyklicznie; część danych jest dostępna w Excelu (appendix tables). ¹²
- OFAC publikuje listy sankcyjne i hash'e zapewniające integralność plików (ważne dla audytu). ¹³
- UNHCR ODP wskazuje CC BY 4.0 (z wyjątkami oznaczonymi) oraz posiada ścieżkę rejestracji API. ¹⁴
- Cloudflare Radar API wymaga tokenu Bearer i ma oficjalny endpoint API. ¹⁵
- OONI API jest „daily updated”, ale **nie jest projektowane do batch-consumption** (engine musi mieć tryb „modest request rate” i/lub batch przez inne kanały). ¹⁶
- IODA (CAIDA) opisuje metodę inferencji outage z wielu źródeł (BGP/darknet/active probing), co czyni ją dobrym niezależnym proxy; istnieje specyfikacja API (endpointy). ¹⁷

Specyfikacja źródeł i poboru danych

Poniżej jest rekomendowany „koszyk danych” (Twoja zasada: wielowarstwowe wskaźniki + triangulacja). Układ jest tak dobrany, żeby zasilić zarówno dashboard „postępu sytuacji”, jak i wejścia do modeli kaskad (SD/sieci), bez wchodzenia w operacyjność.

Warstwa protesty/przemoc (zdarzeniowe, wysokoczęstotliwościowe)

- **Źródło pierwotne:** ACLED ¹⁸ – pobór eventów przez API (kraj=Iran, okno dat), agregacja tygodniowa, rozróżnienie typów zdarzeń, lokalizacja, aktorzy. Kodeks i metodologia opisują, co jest eventem, jakie są pola geograficzne i jak realizowana jest triangulacja źródeł przez ACLED. ¹⁹
- **Uwierzytelnianie:** myACLED + token/cookie workflow (programatycznie: token). ²⁰
- **Ryzyko/TOS:** engine musi implementować paginację i retry, oraz cache ETag/Last-Modified, jeśli dostępne.

Warstwa makro i presja społeczna (niskoczęstotliwościowe, stabilne)

- **Źródło pierwotne:** World Bank ²¹ – Indicators API v2, brak konieczności kluczy, długie szeregi czasowe. ¹¹
- Typowo CPI/inflacja/wybrane wskaźniki (konkretny koszyk wskaźników: **nieokreślone** – warianty poniżej).
- **Warianty koszyka** (gdy nieokreślone):
 - wariant minimalny: CPI (lub GDP deflator), populacja (dla per-capita), ewentualnie bezrobocie jeśli kompletne;
 - wariant rozszerzony: saldo rachunku bieżącego, handel/PKB, urbanizacja, itp.
- **Uwaga:** to są dane miesięczne/roczne → w Twoim standardzie: step-hold do tygodnia + jawne oznaczenie „warstwa tła”.

Warstwa handel i zdolność importowo-eksportowa

- **Źródło pierwotne:** UN Comtrade (nowa platforma i API; dostęp i tokeny bywają zawile). Katalog API UNSD wskazuje istnienie Comtrade API i odsyła do portalu. ²²
- **Dostęp/tokeny i limity:** biblioteki klienckie (np. comtrade) streszczają, że token zwiększa limity (guest vs token) i wskazują ograniczenia query; to trzeba traktować jako „operacyjne constraints” engine’u (throttling + batching). ²³

- **Stan szczegółów:** część parametrów (jakie kategorie HS, jakie flows i częstotliwość) jest **nieokreślona** → engine powinien wspierać:
- „bulk-by-period” (miesiąc/rok w paczkach),
- cache wyników,
- limitowanie liczby partnerów/kodów na request.

Warstwa ropa/energia

- **Źródło:** OPEC ²⁴ – MOMR i załączniki tabelaryczne; OPEC komunikuje dostępność tabel w Excelu (appendix tables) i publikuje harmonogram wydań. ²⁵
- **Dostęp:** część archiwów bywa za loginem w digital publications, ale strona OPEC zawiera publiczne pliki i appendices. ²⁶
- **Nieokreślone:** które konkretnie tabele/seria ma zasilać **OilRevIndex**. Warianty:
- produkcja (Iran / OPEC secondary sources) + benchmark cena ropy (osobne źródło) → indeks przychodów jako proxy,
- same „appendix tables” (jeśli dają potrzebne pola).

Warstwa sankcje i presja zewnętrzna

- **Źródło pierwotne:** OFAC ²⁷ – listy (SDN i non-SDN) + hash values dla pewności integralności plików; to jest „gold standard” audytu poboru. ²⁸
- Engine powinien pobierać pliki (CSV/XML/ZIP), zapisywać raw bytes i hash, następnie parsować do tabeli „sanctions events” oraz budować miesięczny/tygodniowy indeks intensywności (wg Twojej wagi sektorowej w standardzie).

Warstwa kontrola informacji, blackout, łączność

- **Strukturalne:** Freedom House ²⁹ – Freedom on the Net ma udokumentowaną metodologię scoringu (0–100, kategorie) oraz raporty krajowe. ³⁰
- Dostęp do „download data” bywa przez kontakt; engine traktuje to jako wolnozmienny baseline (roczny). ³¹
- **Zdarzeniowe/telemetryczne:**
- Cloudflare ³² Radar API – wymaga tokenu; nadaje się do serii „connectivity disruption / traffic anomalies”, zależnie od endpointów. ³³
- OONI ³⁴ API – dane mierzone globalnie, ale API nie jest do batch; engine musi mieć tryb ograniczonego tempa i/lub offline mining. ¹⁶
- CAIDA ³⁵ IODA – multi-sygnałowe wykrywanie outage (BGP, darknet, active probing) i specyfikacja endpointów API; dobry niezależny proxy. ¹⁷
- NetBlocks COST: przydatne jako proxy kosztu, ale programatyczny pobór i licencja jako dataset do integracji jest często **nieokreślona** (COST to narzędzie-estymator; użyte jako źródło „pośrednie”, nie surowa telemetria). ³⁶

Warstwa humanitarna i migracyjna

- **Źródło:** UNHCR ³⁷ Operational Data Portal: udokumentowana licencja CC BY 4.0 (z wyjątkami) i ścieżka API registration. ¹⁴
- Engine powinien pobierać: migracje/uchodźcy/IDP (zależnie od dostępnych endpointów dla Iranu i/lub regionu), a potem normalizować per-capita i log1p.

Projekt pipeline: od raw do final_model_input

Poniższy projekt implementuje dokładnie Twój standard: równoległe warstwy danych + harmonizacja czasu + transformacje rozkładu + normalizacja 0-1 + budowa zmiennych pośrednich i latentnych z jawnymi wagami.

Struktura danych w repo (proponowana, normatywna)

Wbuduj do **DonkeyJLove/SymulacjaKaskadySieciowej** nowy moduł `data_engine/` oraz katalog `data/`:

```
data/
  registry/
    datasets.yaml          # definicje źródeł + licencje + tryby
  poboru
    variables.yaml         # mapowanie: dataset -> komponent ->
  zmienna modelu
    raw/
      <dataset_id>/
        <run_ts>__sha256=<...>.<ext>[.gz]
    clean/
      <dataset_id>/part-*.parquet
    transformed/
      <dataset_id>/part-*.parquet
    normalized/
      <dataset_id>/part-*.parquet
    model_inputs/
      iran_weekly.parquet
      iran_weekly_metadata.json
    manifests/
      runs/
        <run_id>.json      # run-manifest, request params, wersje,
  seedy
  qa/
    expectations/          # reguły jakości (pandera/great-
expectations lub własne)
    reports/
      <run_id>_qa.json
```

Tę strukturę dopinasz do istniejącego `outputs/` symulacji (nie mieszasz danych obserwacyjnych z artefaktami symulacji).

Koperta zdarzeń i tożsamość (AID jako wzorzec)

Z repo `sbom` bierzesz idiom: *koperta zdarzenia + AID + event_type*. Dla data-engine proponuję analogię:

- `aid.app_id = "iran-data-engine"`
- `aid.env = "prod" | "lab"`
- `aid.vcs_ref = <commit>` (automatycznie z Gita)

- `event_type = "raw_fetch" | "clean" | "transform" | "normalize" | "model_input" | "qa_gate"`

To daje audyt: *kto, kiedy, jak pobrał, czym przetworzył, co wyszło* – w tej samej filozofii „model do wnioskowania, nie archiwum”. 7 6

Mermaid: pipeline eksperymentu danych (pobór → normalizacja → wejście do SD)

flowchart TD

```
A[Rejestr datasets.yaml + variables.yaml] --> B[Pobór: fetchers per źródło]
B --> C[RAW store: immutable bytes + sha256 + request params]
C --> D[Walidacja techniczna: schema/typy/range + rate-limit logs]
D --> E[Harmonizacja czasu: dzień/miesiąc/rok -> tydzień]
E --> F[Clean: braki/outliery + flagi is_imputed/imputation_method]
F --> G[Transform: log1p / robust_z / trend-relative]
G --> H[Normalize: P10-P90 clip / sigmoid(robust_z/k)]
H --> I[Budowa zmiennych B i C: indeksy i latent_proxy_index]
I --> J[Model inputs: iran_weekly.parquet + metadata]
J --> K[Kalibracja/porównanie: SymulacjaKaskadySieciowej]
K --> L[Raport QA + gate: pass/fail + alerty]
```

Harmonizacja czasu do tygodnia (Twoje reguły jako implementacja)

Engine implementuje trzy osobne „resamplery”:

- **dzienne → tygodniowe**: `sum` dla counts (np. protesty), `mean` / `sum-weighted` dla intensywności;
- **miesięczne → tygodniowe**: dwie wersje równoległe: `step_hold` i `linear_interpolated`, z domyślną selekcją `step_hold` (zgodnie z Twoim standardem);
- **roczne/kwartalne → tygodniowe**: baseline strukturalny + komponent zdarzeniowy z innych źródeł (tj. roczne dane nie mogą być jedynym nośnikiem zmiany tygodniowej).

Uwaga spójności: repo SymulacjaKaskadySieciowej jawnie zakłada **krok tygodniowy** i normalizację wielu zmiennych do `[0, 1]`, więc ta część musi być deterministyczna, inaczej rozjeżdża się kalibracja i porównywalność scenariuszy. 3

Braki danych i outliery (standard → kod)

W Twoim standardzie kluczowe jest rozdzielenie: - brak krótki losowy (interpolacja dla continuous), - brak blokowy (flagowanie jakości, nie „udawanie danych”), - brak strukturalny (budowa proxy/latent index, nie imputacja „z powietrza”).

Outliery: dla eventów zwykle zachować (to może być realny szok), a dla makro/handlu detekcja robust: rolling median + MAD i reguła manual review przy `|robust_z| > 5`.

Engine musi zawsze tworzyć metadane: `is_imputed`, `imputation_method`, `quality_flag`, `quality_proxy`, `latent_flag`.

Normalizacja do 0-1 (standard → funkcje)

W engine definiujesz 4 „kanoniczne” normalizery (Twoje wzory jako funkcje biblioteczne):

1. **events_weekly_norm**: per-capita → smoothing EWMA/mean → percentylowe skalowanie P10–P90 z clip.
2. **macro_trade_oil_norm**: log1p (opcjonalnie) → robust_z (median/MAD) albo trend-relative → sigmoid(robust_z/k).
3. **sanctions_norm**: ważona suma typów sankcji → rolling mean 3m → clip / P95.
4. **humanitarian_norm**: per-capita → log1p → P10–P90 clip.

Każda zmienna musi mieć jawnie określony `orientation` (wyżej=gorzej vs wyżej=lepiej) oraz `semantic_direction` przy eksporcie do modelu.

Szkielet implementacji i integracja z SymulacjaKaskadySieciowej

Ta sekcja daje Ci „gotowy do commit” plan implementacyjny (bez udzielania instrukcji operacyjnych do działań szkodliwych; wyłącznie inżynieria danych + badania odporności).

Interfejs engine’u (Python) – minimalny, ale absolutny

Zaprojektuj package:

```
src/abstrakcyjna_symulacja_kaskady_sieciowej/  
  data_engine/  
    __init__.py  
    registry.py          # DatasetSpec/VariableSpec, load YAML, walidacja  
  fetchers/  
    acled.py  
    worldbank.py  
    comtrade.py  
    opec.py  
    ofac.py  
    unhcr.py  
    connectivity.py      # cloudflare/ooni/ioda (oddzielone adapterami)  
  processing/  
    harmonize_time.py  
    clean.py  
    transform.py  
    normalize.py  
    compose_indices.py  
  qa/  
    checks.py  
    gates.py  
  cli.py
```

Powiąz to z istniejącą semantyką configów w repo: analogiczny mechanizm flattening/walidacji jak w `config.py` (default → YAML → CLI overrides). 4

Pseudokod: registry + run-manifest (absolutny audyt)

```
# registry.py (pseudokod)
@dataclass
class DatasetSpec:
    id: str
    source_primary: str
    license: str
    auth: dict          # {"type": "oauth"|"api_key"|"none", "env_vars":
[...]
    schedule: str       # "daily"|"weekly"|"monthly"|"annual"
    raw_format: str     # "json"|"csv"|"zip"|"pdf"...
    fetcher: str        # entrypoint
    rate_limit: dict    # {"rps": 1, "per_hour": 100} etc
    notes: str

@dataclass
class VariableSpec:
    name: str
    class_: str         # A/B/C
    orientation: str    # higher_is_worse|higher_is_better
    frequency_model: str # weekly
    components: list    # list of component series + weights (jawne)
    transform: str
    normalization: str
    semantic_direction: str

def run_fetch(dataset_id, since, until, run_context):
    spec = load_dataset_spec(dataset_id)
    run_id = make_run_id(spec, run_context)  # includes commit hash, time,
params
    manifest = {
        "run_id": run_id,
        "dataset_id": dataset_id,
        "code_version": git_commit(),
        "python": sys.version,
        "request_params": {...},
        "started_utc": now(),
    }
    raw_bytes = fetch(spec, params)
    sha = sha256(raw_bytes)
    write_raw(dataset_id, run_id, sha, raw_bytes)
    manifest["raw_sha256"] = sha
    write_manifest(manifest)
```

Dlaczego to jest „absolutne”: (1) masz deterministyczną tożsamość uruchomienia, (2) masz surowe bajty, (3) masz hash, (4) masz wersję kodu i parametry.

Implementacja poboru dla kluczowych źródeł (tylko high-level, defensywnie)

- ACLED: implementujesz klienta zgodnie z przepływem auth i endpointami; obsługujesz `limit` / paginację, bo API ma domyślny limit i wymaga paginacji przy dużych zbiorach. ¹⁰
- World Bank: klient do Indicators API v2 (bez klucza). ¹¹
- OPEC: pobór MOMR/appendix tables z oficjalnych linków (preferuj Excel tables, bo są „data-friendly”). ¹²
- OFAC: pobór list + weryfikacja hash (jeśli publikowane obok pliku) + parsowanie. ¹³
- UNHCR: pobór przez API po rejestracji; przestrzegasz CC BY 4.0 i zapisujesz metadane licencji. ¹⁴
- Cloudflare/OONI/IODA: osobne fetchery z twardym throttlingiem; OONI szczególnie (API „nie do batch”). ³⁸

Integracja danych z symulatorem SD

Repo SymulacjaKaskadySieciowej ma jasną strukturę uruchamiania i katalog `outputs/` na artefakty symulacji. ³

Proponuję dodać wariant trybu:

- `simulate --data-mode scenario` (jak teraz: profile egzogeniczne)
- `simulate --data-mode calibrated --data-snapshot <run_id>` (czerpie `data/model_inputs/iran_weekly.parquet` i mapuje na wejścia `Info`, `Protest`, `Fiscal`, `Sanctions`, `OilRevIndex`, `Displaced` itd.)

Minimalna forma integracji (bez refaktoru core modelu): `model_interface.py` dostaje opcję wczytania szeregu czasowego jako zewnętrznych profili egzogenicznych.

Wykorzystanie wzorców metryk sieciowych (opcjonalnie)

Jeżeli chcesz mierzyć „procesy kaskadowe” także jako **zmiany strukturalne** (np. grafy zależności, łączność, supply/trade network), możesz użyć wzorca z `glitchlab.analysis.graph_metrics` do generowania zestandaryzowanych metryk i artefaktów (pagerank/betweenness/clustering/community). ⁹

Ograniczenia, ryzyka, etyka oraz kluczowe referencje

Ograniczenia i ryzyka (techniczne + metodologiczne)

Najważniejsze ryzyka w takim engine (i jak je gasisz w reżimie absolutnym):

- **Dryf definicji źródeł / zmiany API:** World Bank v1 jest wycofane (musisz używać v2), więc engine musi mieć test „endpoint health” i adaptory wersji. ¹¹
- **Dostęp i tokeny:** ACLED i Cloudflare wymagają auth; Comtrade może zmieniać politykę kluczy/subscription; bez jawnych zmiennych środowiskowych i throttlingu pipeline będzie kruchy. ³⁹
- **Niejednorodne częstotliwości:** tygodniowy SD vs miesięczne/roczne źródła – bez jawnych metod resamplingu (step-hold vs interpolacja) łatwo o artefakty.
- **Latent variables:** zmienne C (morale/lojalność/kohezja elit) muszą być jawnie proxy; w przeciwnym razie model „udaje pomiar”. Repo SymulacjaKaskadySieciowej samo wskazuje, że część stanów jest modelowana przez proxy i dopasowanie historyczne, a nie wprost „mierzalna”. ³

- **Niestabilność normalizacji min-max:** dlatego w Twoim standardzie percentyle P10–P90 i robust MAD; engine ma to „wymuszać”, a nie zostawiać w notebookach.
- **Korelacje wejść:** sankcje/handel/ropa są współzależne; engine powinien zachowywać warianty przetworzeń i umożliwiać testy wrażliwości na założenia (np. alternatywne wagi indeksów, alternatywne normalizacje).

Etyka i dual-use (zasada bezpieczeństwa)

Ten engine jest projektowany jako **narzędzie badawcze i odpornościowe** (monitoring wskaźników + analiza systemów złożonych + kalibracja modeli), nie jako narzędzie operacyjne. W szczególności: - korzysta z **otwartych danych i oficjalnych API**, - nie obejmuje danych taktycznych, identyfikacji celów ani instrukcji działań szkodliwych, - utrzymuje audytowalność (hash-e, manifesty, wersje), co ułatwia kontrolę nadużyć.

Mermaid: timeline wdrożenia (realistyczny plan repo)

```

timeline
    title Plan wdrożenia data-engine w repo (reżim absolutny)
    2026-03-Week1 : Scaffold: data/ + registry YAML + run-manifest + test harness
    2026-03-Week2 : Fetchers v1: World Bank (v2) + OFAC + QA gates (hash/schema)
    2026-03-Week3 : Fetchers v1: ACLED (auth + pagination) + weekly aggregation
    2026-04-Week1 : Trade/Energy: Comtrade adaptor (token + throttling) + OPEC appendix ingestion
    2026-04-Week2 : Connectivity: Cloudflare Radar + OONI + IODA (throttled) + InfoControlShock
    2026-04-Week3 : Compose indices: klasy B/C + wagi + sensitivity wag + export model_inputs
    2026-04-Week4 : Integracja z SD: data-mode calibrated + porównanie scenario vs calibrated
  
```

Kluczowe referencje (źródła pierwotne / oficjalne)

- ACLED ¹⁸ : dokumentacja API i uwierzytelnianie ⁴⁰ oraz codebook/metodologia ⁴¹
- World Bank ²¹ : Indicators API v2 (oficjalna dokumentacja) ¹¹
- OPEC ²⁴ : MOMR + informacja o tabelach appendix w Excelu ²⁵
- OFAC ²⁷ : listy sankcyjne + hash values i standardy danych ²⁸
- UNHCR ³⁷ : ODP + API registration + licencja CC BY 4.0 ¹⁴
- Cloudflare ³² : Radar API (token + endpoint) ³³
- OONI ³⁴ : API (ograniczenie batch) ¹⁶
- CAIDA ³⁵ : IODA (opis systemu) ⁴² i specyfikacja API endpointów ⁴³
- Repo-wzorce (kontrakty danych/artefakty): `sbom` (AID + koperta zdarzenia + event_type) ⁶
⁷ oraz `SymulacjaKaskadySieciowej` (krok tygodniowy, konfiguracje, GSA) ³ ⁵

¹ ¹⁰ ²⁰ ³⁹ Getting started | ACLED

https://acleddata.com/reactivation/api-authentication?utm_source=chatgpt.com

² ⁴⁰ API documentation | ACLED

https://acleddata.com/acled-api-documentation?utm_source=chatgpt.com

- 3 29 **README.md**
<https://github.com/DonkeyJLove/SymulacjaKaskadySieciowej/blob/main/README.md>
- 4 **config.py**
https://github.com/DonkeyJLove/SymulacjaKaskadySieciowej/blob/main/src/abstrakcyjna_symulacja_kaskady_sieciowej/config.py
- 5 **gsa_common.py**
https://github.com/DonkeyJLove/SymulacjaKaskadySieciowej/blob/main/src/abstrakcyjna_symulacja_kaskady_sieciowej/gsa_common.py
- 6 **AID_CONTRACT.md**
https://github.com/DonkeyJLove/sbom/blob/77fbb61e4d583ff4f91cce8c06cdc61e31201c96/AID_CONTRACT.md
- 7 27 **04_DATA_MODEL.md**
https://github.com/DonkeyJLove/sbom/blob/77fbb61e4d583ff4f91cce8c06cdc61e31201c96/docs/04_DATA_MODEL.md
- 8 **docker-compose.lab.yml**
<https://github.com/DonkeyJLove/sbom/blob/77fbb61e4d583ff4f91cce8c06cdc61e31201c96/srodowiska-testowe/docker-compose.lab.yml>
- 9 **graph_metrics.py**
https://github.com/DonkeyJLove/glitchlab/blob/e19a649940ba4b1bea50b10e69622ba8d05a5a61/analysis/graph_metrics.py
- 11 24 **About the Indicators API Documentation – World Bank Data Help Desk**
https://datahelpdesk.worldbank.org/knowledgebase/articles/889392-about-the-indicators-api-documentation?utm_source=chatgpt.com
- 12 25 34 37 **Organization of the Petroleum Exporting Countries**
https://www.opec.org/monthly-oil-market-report.html?utm_source=chatgpt.com
- 13 21 28 **Additional Sanctions Lists | Office of Foreign Assets Control**
https://ofac.treasury.gov/consolidated-sanctions-list-non-sdn-lists/non-sdn-menu-based-sanctions-list-ns-mbs-list?utm_source=chatgpt.com
- 14 **API Registration**
https://data.unhcr.org/en/api/api-registration?utm_source=chatgpt.com
- 15 33 35 38 **Make your first Radar API request · Cloudflare Radar docs**
https://developers.cloudflare.com/radar/get-started/first-request/?utm_source=chatgpt.com
- 16 **OONI API**
https://api.ooni.org/?utm_source=chatgpt.com
- 17 32 42 **IODA - CAIDA**
https://www.caida.org/projects/ioda/?utm_source=chatgpt.com
- 18 22 **UNSD API Catalogue**
https://unstats.un.org/unsd/api/?utm_source=chatgpt.com
- 19 41 **ACLED Codebook | ACLED**
https://acleddata.com/knowledge-base/codebook/?utm_source=chatgpt.com
- 23 **ct_search function - RDocumentation**
https://www.rdocumentation.org/packages/comtradr/versions/0.2.1/topics/ct_search?utm_source=chatgpt.com
- 26 **OPEC Digital Publications - Monthly Oil Market Report**
https://publications.opec.org/momr/archive/download/141/0?utm_source=chatgpt.com
- 30 **Freedom on the Net Research Methodology | Freedom House**
https://freedomhouse.org/reports/freedom-net/freedom-net-research-methodology?utm_source=chatgpt.com

31 Freedom on the Net | Freedom House

https://freedomhouse.org/report/freedom-net?utm_source=chatgpt.com

36 COST: The NetBlocks Cost of Shutdown Tool - NetBlocks

https://netblocks.org/projects/cost?utm_source=chatgpt.com

43 API Specification - CAIDA/ioda-api GitHub Wiki

https://github-wiki-see.page/m/CAIDA/ioda-api/wiki/API-Specification?utm_source=chatgpt.com